GHraisoni INTERNATIONAL SKILLTECH UNIVERSITY

Pune

School of Engineering and Technology

Operating System Lab (25BTPCP402)
B.Tech

LAB MANUAL

Name of the Student: .....

Register No :.....

Class :.....

Year/Sem/Class : ....

This page is left intentionally blank

GHraisoni INTERNATIONAL SKILLTECH UNIVERSITY Pune

School of Engineering and Technology

BONAFIDE CERTIFICATE

Bonafide record of the work done by …….....
……. of ……..
in Operating System Lab (25BTPCP402) , B.Tech during 4th semester in academic year 2025 – 2026.

Staff in-charge Dean

Submitted to the Practical Examination held at GHRISTU on ……

Register Number														

INTERNA EXAMINER **EXTERNAL EXAMINER**

INDEX

Sl.no	Experiment name	Page	Date of Experiment	Date of submission	Marks	Sign
1.	Study of Unix/Linux Basic Commands					
2.	Shell Script to Print Table of a Number					
3.	Menu Driven Calculator Using Shell Script					
4.	Shell Script for Factorial and Greatest Number					
5.	Shell Script for Reverse Number and Even–Odd Array					
6.	Menu Driven Shell Script Using Case Statement					
7.	Process Creation Using `fork()` System Call					
8.	Non-Preemptive CPU Scheduling Algorithms					
9.	Banker's Algorithm for Deadlock Avoidance					
10.	Preemptive CPU Scheduling Algorithms					
11.	Page Replacement Algorithms					
12	Study and Implementation of RAID Operations					
13	Program to Execute Unix Commands					
14	Mini Project on File Handling					
15	Operating Systems Virtual Lab					

EXPERIMENT 1

AIM :

To study and execute basic Unix/Linux general purpose utility commands.

INTRODUCTION :

Unix/Linux operating system provides a variety of general purpose utility commands that help users interact with the system. These commands are used for file and directory management, process monitoring, system information, and user management. Understanding these commands is essential for working efficiently in a Linux environment.

PROCEDURE :

1. Start the Linux Operating System.
2. Open the terminal window.
3. Type the required Unix/Linux commands at the command prompt.
4. Press Enter after each command.
5. Observe the output displayed on the terminal.

Code :

```
pwd
ls
mkdir test
cd test
touch file1.txt
cat file1.txt
cp file1.txt file2.txt
mv file2.txt file3.txt
rm file3.txt
ps
who
date
```

cal
history

WORKSHEET :

1. What is Unix/Linux operating system?
2. What is the use of ls command?
3. Explain the purpose of pwd command.
4. What is the difference between cp and mv commands?
5. What is the use of ps command?

CONCLUSION :

Thus, the experiment was performed successfully. We studied and executed various Unix/Linux general purpose utility commands and understood their usage and output in the Linux environment.

EXPERIMENT 2

AIM :

To write a shell script program to print the multiplication table of a given number.

INTRODUCTION :

Shell scripting is a powerful feature of the Unix/Linux operating system that allows users to automate tasks by writing a sequence of commands in a file. In this experiment, a shell script is used to generate the multiplication table of a given number, which helps in understanding user input, looping statements, and arithmetic operations in shell scripting.

PROCEDURE :

1. Start the Linux Operating System.
2. Open the terminal window.
3. Create a new shell script file using vi or nano editor.
4. Write the shell script code to accept a number and display its table.
5. Save the file.
6. Provide execute permission using chmod +x filename.sh.
7. Execute the script using ./filename.sh.
8. Observe the output on the terminal.

Code :

```
echo "Enter a number"
read n
for i in {1..10}
do
 echo "$n x $i = $((n*i))"
done
```

WORKSHEET :

1. What is shell scripting?
2. What is the use of read command in shell script?
3. Explain the for loop used in the program.
4. What is the purpose of ?
5. How are arithmetic operations performed in shell script?

CONCLUSION :

Thus, the experiment was carried out successfully. The shell script to print the multiplication table of a given number was executed and the correct output was obtained, helping us understand looping and arithmetic operations in shell scripting.

EXPERIMENT 3

AIM :

To develop a menu driven calculator using shell scripting.

INTRODUCTION :

A menu driven program allows the user to select an operation from a list of available options. Shell scripting supports decision-making statements such as case, which makes it suitable for developing menu driven applications. In this experiment, a calculator is developed using shell scripting to perform basic arithmetic operations like addition, subtraction, multiplication, and division.

PROCEDURE :

1. Start the Linux Operating System.
2. Open the terminal window.
3. Create a shell script file using vi or nano editor.
4. Write the shell script to display a menu and accept user choice.
5. Use case statement to perform the selected arithmetic operation.
6. Save the file.
7. Give execution permission using chmod +x filename.sh.
8. Execute the program and observe the result.

Code :

```
echo "Menu"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"

read ch
```

```
echo "Enter two numbers"
read a b

case $ch in
1) echo "Addition = $((a+b))";;
2) echo "Subtraction = $((a-b))";;
3) echo "Multiplication = $((a*b))";;
4) echo "Division = $((a/b))";;
*) echo "Invalid Choice";;
esac
```

WORKSHEET :

1. What is a menu driven program?
2. Explain the use of case statement in shell scripting.
3. How is user input taken in shell script?
4. What happens if an invalid choice is entered?
5. How is division performed in shell script?

CONCLUSION :

Thus, the experiment was completed successfully. A menu driven calculator using shell scripting was developed and executed, and basic arithmetic operations were performed correctly.

EXPERIMENT 4

AIM :

a) To find the factorial of a given number using shell script.
b) To find the greatest of three numbers using shell script.

INTRODUCTION :

Shell scripting allows the use of conditional statements and looping constructs to solve logical problems. Factorial calculation helps in understanding looping structures, while finding the greatest of three numbers helps in understanding conditional statements. This experiment demonstrates the use of for loop and if-else conditions in shell scripting.

PROCEDURE :

1. Start the Linux Operating System.
2. Open the terminal window.
3. Create a shell script file using vi or nano.
4. Write the shell script code for factorial and greatest number logic.
5. Save the file properly.
6. Provide execution permission using chmod +x filename.sh.
7. Execute the script using ./filename.sh.
8. Observe the output.

Code :

```
# Factorial of a number
echo "Enter a number for factorial"
read n
fact=1
for ((i=1; i<=n; i++))
do
```

```
 fact=$((fact*i))
done
echo "Factorial of $n is $fact"


# Greatest of three numbers
echo "Enter three numbers"
read a b c
if [ $a -gt $b ] && [ $a -gt $c ]
then
 echo "$a is greatest"
elif [ $b -gt $c ]
then
 echo "$b is greatest"
else
 echo "$c is greatest"
fi
```

WORKSHEET :

1. What is factorial?
2. Which loop is used to calculate factorial in this program?
3. Explain the use of if-elif-else statement.
4. What does the -gt operator indicate?
5. Can this program work for negative numbers? Why?

CONCLUSION :

Thus, the experiment was performed successfully. The factorial of a given number and the greatest of three numbers were calculated correctly using shell scripting, which helped in understanding loops and conditional statements.

EXPERIMENT 5

AIM :

a) To print the reverse of a given number using shell script.
b) To find even and odd numbers from a given array using shell script.

INTRODUCTION :

Shell scripting supports arithmetic operations, loops, and conditional statements which can be used to manipulate numbers and arrays. Reversing a number helps in understanding while loops and arithmetic operations, while identifying even and odd numbers from an array helps in understanding arrays and conditional checking in shell scripting.

PROCEDURE :

1. Start the Linux Operating System.
2. Open the terminal window.
3. Create a shell script file using vi or nano editor.
4. Write the shell script code for reversing a number and checking even–odd numbers.
5. Save the file.
6. Provide execution permission using chmod +x filename.sh.
7. Execute the script using ./filename.sh.
8. Observe the output displayed on the terminal.

Code :

```
# Reverse of a number
echo "Enter a number"
read n
rev=0
while [ $n -gt 0 ]
do
```

```
 r=$((n%10))
 rev=$((rev*10+r))
 n=$((n/10))
done
echo "Reverse of the number is $rev"


# Even and Odd numbers from array
arr=(10 15 20 25 30)
echo "Even and Odd numbers:"
for i in ${arr[@]}
do
 if [ $((i%2)) -eq 0 ]
 then
  echo "$i is Even"
 else
  echo "$i is Odd"
 fi
done
```

WORKSHEET :

1. Which loop is used to reverse the number?
2. Explain the logic used for reversing a number.
3. What is an array in shell scripting?
4. How is even or odd number identified in the program?
5. What does % operator represent?

CONCLUSION :

Thus, the experiment was completed successfully. The reverse of a given number was printed and even and odd numbers were identified from an array using shell scripting, helping to understand loops, arrays, and conditional statements.

EXPERIMENT 6

AIM :

To develop a menu driven shell script using switch case to perform factorial, greatest of three numbers, reverse of a number, and even–odd operations.

INTRODUCTION :

A menu driven program allows users to select an operation from multiple options. Shell scripting supports decision-making using the case statement, which acts as a switch case. This experiment demonstrates how multiple operations such as factorial calculation, finding the greatest number, reversing a number, and checking even–odd numbers can be implemented in a single shell script using a menu driven approach.

PROCEDURE :

1. Start the Linux Operating System.
2. Open the terminal window.
3. Create a shell script file using vi or nano editor.
4. Write the shell script code using case statement for menu options.
5. Save the file.
6. Provide execution permission using chmod +x filename.sh.
7. Execute the program using ./filename.sh.
8. Select the desired option from the menu and observe the output.

Code :

```
echo "Menu"
echo "1. Factorial"
```

```
echo "2. Greatest of three numbers"
echo "3. Reverse of a number"
echo "4. Even and Odd from array"

read ch

case $ch in
1)
 echo "Enter number"
 read n
 fact=1
 for ((i=1;i<=n;i++))
 do
  fact=$((fact*i))
 done
 echo "Factorial = $fact"
 ;;
2)
 echo "Enter three numbers"
 read a b c
 if [ $a -gt $b ] && [ $a -gt $c ]
 then
  echo "$a is greatest"
 elif [ $b -gt $c ]
 then
  echo "$b is greatest"
 else
  echo "$c is greatest"
 fi
 ;;
3)
 echo "Enter number"
 read n
 rev=0
 while [ $n -gt 0 ]
 do
  r=$((n%10))
  rev=$((rev*10+r))
  n=$((n/10))
```

```
 done
 echo "Reverse = $rev"
 ;;
4)
 arr=(5 10 15 20)
 for i in ${arr[@]}
 do
  if [ $((i%2)) -eq 0 ]
  then
   echo "$i is Even"
  else
   echo "$i is Odd"
  fi
 done
 ;;
*)
 echo "Invalid choice"
 ;;
esac
```

WORKSHEET :

1. What is a menu driven program?
2. Explain the use of case statement in shell scripting.
3. How does the program switch between different operations?
4. What is the advantage of menu driven programs?
5. Which operations are performed in this experiment?

CONCLUSION :

Thus, the experiment was performed successfully. A menu driven shell script using switch case was developed to perform multiple operations, which helped in understanding conditional statements and modular programming in shell scripting.